

Shortest Path Algorithms

Sean

August 27, 2026

Outline

- From BFS to weighted paths
- Dijkstra's algorithm
- Special cases
- Negative edges
- Choosing the right algorithm



What changes after Graph Basics?

- In Graph Basics, BFS found shortest paths when every move cost 1



What changes after Graph Basics?

- In Graph Basics, BFS found shortest paths when every move cost 1
- Then distance meant the fewest number of edges



What changes after Graph Basics?

- In Graph Basics, BFS found shortest paths when every move cost 1
- Then distance meant the fewest number of edges
- Real problems often attach different costs to different edges



What changes after Graph Basics?

- In Graph Basics, BFS found shortest paths when every move cost 1
- Then distance meant the fewest number of edges
- Real problems often attach different costs to different edges
- This lecture asks: how do we keep the shortest-path idea once edges have weights?



1 From BFS to weighted paths

2 Dijkstra's algorithm

3 Special cases

4 Negative edges

5 Choosing the right algorithm



From BFS to weighted paths

What BFS already gives us

- BFS explores vertices by increasing edge count

What BFS already gives us

- BFS explores vertices by increasing edge count
- The first visit to a vertex is already optimal



What BFS already gives us

- BFS explores vertices by increasing edge count
- The first visit to a vertex is already optimal
- We can store $\text{dist}[v]$ and $\text{parent}[v]$ while exploring



What BFS already gives us

- BFS explores vertices by increasing edge count
- The first visit to a vertex is already optimal
- We can store $\text{dist}[v]$ and $\text{parent}[v]$ while exploring
- The hidden assumption is that every edge contributes the same cost

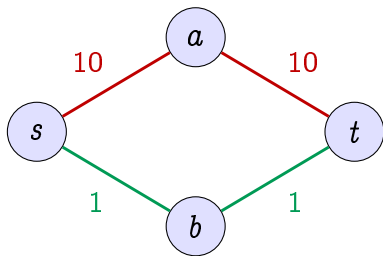


Weighted shortest paths

- A weighted edge has a cost, such as distance, time, or price
- The length of a path is the sum of its edge weights
- We now want the minimum total cost, not the fewest edges
- The graph can still be directed or undirected



Why BFS stops working



- Both routes use 2 edges
- Top route cost: 20
- Bottom route cost: 2
- Edge count no longer tells the whole story

The new goal

- Keep the same graph model

The new goal

- Keep the same graph model
- Replace “fewest edges” with “smallest total weight”

The new goal

- Keep the same graph model
- Replace “fewest edges” with “smallest total weight”
- Replace layer order with cost order

The new goal

- Keep the same graph model
- Replace “fewest edges” with “smallest total weight”
- Replace layer order with cost order
- This leads to Dijkstra’s algorithm when all edge weights are non-negative

1 From BFS to weighted paths

2 Dijkstra's algorithm

3 Special cases

4 Negative edges

5 Choosing the right algorithm



Dijkstra's algorithm

The relaxation idea

- Suppose we already know a path to u with cost $\text{dist}[u]$

The relaxation idea

- Suppose we already know a path to u with cost $\text{dist}[u]$
- Following edge $u \rightarrow v$ with weight w gives candidate cost $\text{dist}[u] + w$



The relaxation idea

- Suppose we already know a path to u with cost $\text{dist}[u]$
- Following edge $u \rightarrow v$ with weight w gives candidate cost $\text{dist}[u] + w$
- If that candidate is smaller than $\text{dist}[v]$, improve $\text{dist}[v]$



The relaxation idea

- Suppose we already know a path to u with cost $\text{dist}[u]$
- Following edge $u \rightarrow v$ with weight w gives candidate cost $\text{dist}[u] + w$
- If that candidate is smaller than $\text{dist}[v]$, improve $\text{dist}[v]$
- This update is called **relaxing** the edge



Relaxing one edge

```
1  if (dist[u] + w < dist[v]) {  
2      dist[v] = dist[u] + w;  
3      parent[v] = u;  
4  }
```

Dijkstra's algorithm

- Start with $\text{dist}[\text{start}] = 0$; every other distance is infinity

Dijkstra's algorithm

- Start with $\text{dist}[\text{start}] = 0$; every other distance is infinity
- Always process the not-yet-final vertex with smallest current distance



Dijkstra's algorithm

- Start with $\text{dist}[\text{start}] = 0$; every other distance is infinity
- Always process the not-yet-final vertex with smallest current distance
- Relax every outgoing edge from that vertex



Dijkstra's algorithm

- Start with $\text{dist}[\text{start}] = 0$; every other distance is infinity
- Always process the not-yet-final vertex with smallest current distance
- Relax every outgoing edge from that vertex
- A min-priority queue gives the next smallest tentative distance efficiently

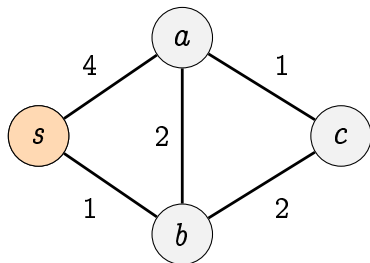


Dijkstra's algorithm

- Start with $\text{dist}[\text{start}] = 0$; every other distance is infinity
- Always process the not-yet-final vertex with smallest current distance
- Relax every outgoing edge from that vertex
- A min-priority queue gives the next smallest tentative distance efficiently
- Non-negative weights ensure finalized distances never need to decrease later

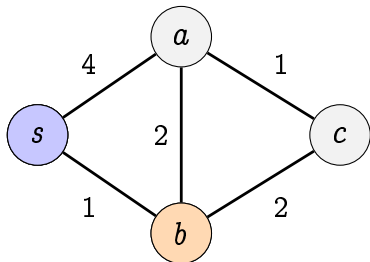


Dijkstra trace



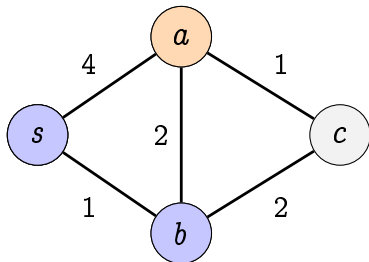
- Start: $d(s) = 0$

Dijkstra trace



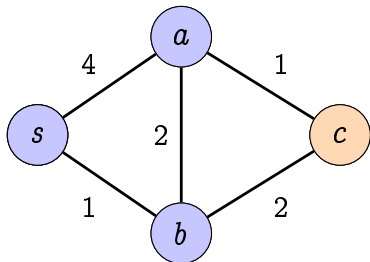
- Start: $d(s) = 0$
- After s :
 $d(a) = 4, d(b) = 1$

Dijkstra trace



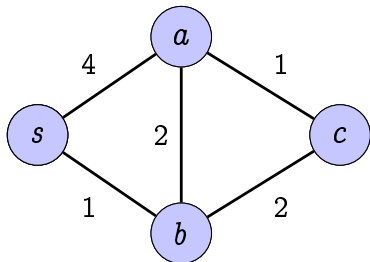
- Start: $d(s) = 0$
- After s :
 $d(a) = 4$, $d(b) = 1$
- After b :
 $d(a) = 3$, $d(c) = 3$

Dijkstra trace



- Start: $d(s) = 0$
- After s :
 $d(a) = 4, d(b) = 1$
- After b :
 $d(a) = 3, d(c) = 3$
- After a : $d(c)$ stays 3

Dijkstra trace



- Start: $d(s) = 0$
- After s :
 $d(a) = 4, d(b) = 1$
- After b :
 $d(a) = 3, d(c) = 3$
- After a : $d(c)$ stays 3
- Final distances:
0, 3, 1, 3

Dijkstra code

```
1  const long long INF = 4e18;
2  vector<long long> dist(n + 1, INF);
3  vector<int> parent(n + 1, -1);
4  using State = pair<long long,int>;
5  priority_queue<State, vector<State>, greater<State>> pq;
6
7  dist[start] = 0;
8  pq.push({0, start});
9
10 while (!pq.empty()) {
11     auto [d, u] = pq.top();
12     pq.pop();
13     if (d != dist[u]) continue;
14
15     for (auto [v, w] : adj[u]) {
16         if (dist[u] + w < dist[v]) {
17             dist[v] = dist[u] + w;
18             parent[v] = u;
19             pq.push({dist[v], v});
20         }
21     }
22 }
```



Why stale entries appear

- A vertex can receive a better distance after an older heap entry was inserted

Why stale entries appear

- A vertex can receive a better distance after an older heap entry was inserted
- Instead of deleting old heap entries, we leave them there

Why stale entries appear

- A vertex can receive a better distance after an older heap entry was inserted
- Instead of deleting old heap entries, we leave them there
- When popped, if $(d \neq \text{dist}[u])$ continue;
discards stale work



Why stale entries appear

- A vertex can receive a better distance after an older heap entry was inserted
- Instead of deleting old heap entries, we leave them there
- When popped, if $(d \neq \text{dist}[u])$ continue; discards stale work
- This keeps the implementation simple and still efficient



Why Dijkstra works

- The next chosen vertex has the smallest tentative distance



Why Dijkstra works

- The next chosen vertex has the smallest tentative distance
- Any alternative path reaching it later must first pass through a vertex with distance at least as large



Why Dijkstra works

- The next chosen vertex has the smallest tentative distance
- Any alternative path reaching it later must first pass through a vertex with distance at least as large
- Non-negative edges cannot make that later route cheaper

Why Dijkstra works

- The next chosen vertex has the smallest tentative distance
- Any alternative path reaching it later must first pass through a vertex with distance at least as large
- Non-negative edges cannot make that later route cheaper
- So once a vertex is selected, its distance is final

Recovering a weighted path

```
1  vector<int> path;
2  for (int cur = target; cur != -1; cur = parent[cur]) {
3      path.push_back(cur);
4  }
5  reverse(path.begin(), path.end());
```

- Store $\text{parent}[v] = u$ whenever relaxation improves $\text{dist}[v]$
- The recovery step is the same idea as BFS path recovery



Example: Shortest Routes I

CSES: Shortest Routes I

Find the minimum cost from city 1 to every city in a directed weighted graph.

Example: Shortest Routes I

CSES: Shortest Routes I

Find the minimum cost from city 1 to every city in a directed weighted graph.

- Edge weights are non-negative

Example: Shortest Routes I

CSES: Shortest Routes I

Find the minimum cost from city 1 to every city in a directed weighted graph.

- Edge weights are non-negative
- One source, all destinations

Example: Shortest Routes I

CSES: Shortest Routes I

Find the minimum cost from city 1 to every city in a directed weighted graph.

- Edge weights are non-negative
- One source, all destinations
- This is the standard Dijkstra setting



Example: Shortest Routes I

CSES: Shortest Routes I

Find the minimum cost from city 1 to every city in a directed weighted graph.

- Edge weights are non-negative
- One source, all destinations
- This is the standard Dijkstra setting
- Use `long long`; total path cost can exceed `int`



1 From BFS to weighted paths

2 Dijkstra's algorithm

3 Special cases

4 Negative edges

5 Choosing the right algorithm



Special cases

When BFS still wins

- If every edge weight is exactly 1, use BFS

When BFS still wins

- If every edge weight is exactly 1, use BFS
- BFS is simpler and runs in $\mathcal{O}(n + m)$



When BFS still wins

- If every edge weight is exactly 1, use BFS
- BFS is simpler and runs in $\mathcal{O}(n + m)$
- Dijkstra would also work, but it adds an unnecessary heap



When BFS still wins

- If every edge weight is exactly 1, use BFS
- BFS is simpler and runs in $\mathcal{O}(n + m)$
- Dijkstra would also work, but it adds an unnecessary heap
- Use the simplest algorithm whose assumptions match the graph



0-1 BFS

- If every edge weight is either 0 or 1, use a deque



0-1 BFS

- If every edge weight is either 0 or 1, use a deque
- Weight 0: push the neighbor to the front



0-1 BFS

- If every edge weight is either 0 or 1, use a deque
- Weight 0: push the neighbor to the front
- Weight 1: push the neighbor to the back

0-1 BFS

- If every edge weight is either 0 or 1, use a deque
- Weight 0: push the neighbor to the front
- Weight 1: push the neighbor to the back
- The deque keeps vertices in nondecreasing distance order



0-1 BFS

- If every edge weight is either 0 or 1, use a deque
- Weight 0: push the neighbor to the front
- Weight 1: push the neighbor to the back
- The deque keeps vertices in nondecreasing distance order
- Runtime is $\mathcal{O}(n + m)$



0-1 BFS code

```
1  deque<int> dq;
2  vector<bool> done(n + 1, false);
3  dist[start] = 0;
4  dq.push_front(start);
5
6  while (!dq.empty()) {
7      int u = dq.front();
8      dq.pop_front();
9      if (done[u]) continue;
10     done[u] = true;
11
12     for (auto [v, w] : adj[u]) {
13         if (dist[u] + w < dist[v]) {
14             dist[v] = dist[u] + w;
15             if (w == 0) dq.push_front(v);
16             else dq.push_back(v);
17         }
18     }
19 }
```



1 From BFS to weighted paths

2 Dijkstra's algorithm

3 Special cases

4 Negative edges

5 Choosing the right algorithm

Negative edges

What negative edges break

- Dijkstra assumes a finalized distance can never improve later

What negative edges break

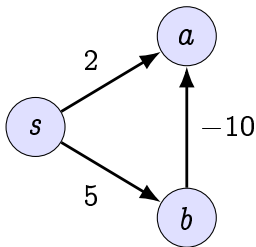
- Dijkstra assumes a finalized distance can never improve later
- A negative edge can create a cheaper route after a vertex was already processed

What negative edges break

- Dijkstra assumes a finalized distance can never improve later
- A negative edge can create a cheaper route after a vertex was already processed
- So Dijkstra is not valid when negative edge weights are allowed



Dijkstra counterexample



- Direct path to a :
cost 2
- Path $s \rightarrow b \rightarrow a$:
cost -5
- Finalizing a too early
is wrong

Bellman–Ford

- Relax every edge repeatedly

Bellman–Ford

- Relax every edge repeatedly
- After the k -th pass, all shortest paths using at most k edges are known

Bellman–Ford

- Relax every edge repeatedly
- After the k -th pass, all shortest paths using at most k edges are known
- After $n - 1$ passes, every simple shortest path is covered



Bellman–Ford

- Relax every edge repeatedly
- After the k -th pass, all shortest paths using at most k edges are known
- After $n - 1$ passes, every simple shortest path is covered
- A further improvement on pass n means a reachable negative cycle exists



Bellman–Ford

- Relax every edge repeatedly
- After the k -th pass, all shortest paths using at most k edges are known
- After $n - 1$ passes, every simple shortest path is covered
- A further improvement on pass n means a reachable negative cycle exists
- Runtime is $\mathcal{O}(nm)$



Bellman–Ford code

```
1  vector<long long> dist(n + 1, INF);
2  dist[start] = 0;
3
4  bool negative_cycle = false;
5  for (int iter = 1; iter <= n; iter++) {
6      bool changed = false;
7      for (auto [u, v, w] : edges) {
8          if (dist[u] == INF) continue;
9          if (dist[u] + w < dist[v]) {
10             dist[v] = dist[u] + w;
11             changed = true;
12         }
13     }
14     if (!changed) break;
15     if (iter == n) negative_cycle = true;
16 }
```

- A change on pass n means a reachable negative cycle exists



Negative cycles

- A negative cycle has total weight below 0

Negative cycles

- A negative cycle has total weight below 0
- Repeating it can make path cost smaller forever

Negative cycles

- A negative cycle has total weight below 0
- Repeating it can make path cost smaller forever
- Then a finite shortest path may not exist

Negative cycles

- A negative cycle has total weight below 0
- Repeating it can make path cost smaller forever
- Then a finite shortest path may not exist
- Bellman–Ford can detect this by checking for one more relaxation

What if we need every pair?

- So far, each algorithm starts from one source vertex

What if we need every pair?

- So far, each algorithm starts from one source vertex
- Some problems ask for shortest paths between every pair of vertices



What if we need every pair?

- So far, each algorithm starts from one source vertex
- Some problems ask for shortest paths between every pair of vertices
- Floyd–Warshall solves the all-pairs version in $\mathcal{O}(n^3)$



What if we need every pair?

- So far, each algorithm starts from one source vertex
- Some problems ask for shortest paths between every pair of vertices
- Floyd–Warshall solves the all-pairs version in $\mathcal{O}(n^3)$
- We will return to it later when dynamic programming is the main idea



What if we know the target?

- Dijkstra explores outward without knowing where the goal is

What if we know the target?

- Dijkstra explores outward without knowing where the goal is
- A* adds a heuristic estimate of the remaining distance

What if we know the target?

- Dijkstra explores outward without knowing where the goal is
- A* adds a heuristic estimate of the remaining distance
- With a good heuristic that never overestimates, it can search much less than Dijkstra



What if we know the target?

- Dijkstra explores outward without knowing where the goal is
- A* adds a heuristic estimate of the remaining distance
- With a good heuristic that never overestimates, it can search much less than Dijkstra
- We will leave heuristic search for another time

1 From BFS to weighted paths

2 Dijkstra's algorithm

3 Special cases

4 Negative edges

5 Choosing the right algorithm

Choosing the right algorithm

Shortest-path decision table

Edge weights	Use	Runtime
All 1	BFS	$\mathcal{O}(n + m)$
Only 0 or 1	0–1 BFS	$\mathcal{O}(n + m)$
Non-negative	Dijkstra	$\mathcal{O}((n + m) \log n)$
May be negative	Bellman–Ford	$\mathcal{O}(nm)$

Common mistakes

- Using BFS when edge weights differ
- Using Dijkstra when negative edges are possible
- Storing distances in `int` when path sums need `long long`
- Forgetting to skip stale priority-queue entries
- Treating directed edges as undirected



Practice problems

- CSES: Shortest Routes I – one-source Dijkstra
- CSES: Investigation – shortest paths plus extra information
- CSES: Flight Discount – Dijkstra with state expansion
- CSES: Cycle Finding – negative-cycle detection



The End

Thanks for listening.

© 2026 Sean. Please do not repost or redistribute without permission.

